

Call by Value vs. Call by Reference

1. Call by Value (Go's Default Behavior)

This means when you pass a variable into a function, Go creates a complete, independent **copy** of that data and hands the copy to the function's parameter.

- **How it works:** If you have $x = 2$ in your main function and pass it into a function `foo(y)`, `y` becomes a copy holding the value 2. If `foo` modifies `y` (e.g., `y = y + 1`), your original `x` back in main remains untouched.
- **The Advantage (Data Safety):** This guarantees strict **Data Encapsulation**. If a function contains a bug and corrupts its data, it only ruins its own isolated copy. The main state of your application remains perfectly safe.
- **The Disadvantage (Performance):** If you are passing a massive data structure (like a Slice with 100,000 items), Go has to physically duplicate every single piece of that data. This wastes CPU cycles and consumes extra RAM.

2. Call by Reference (Using Pointers)

While Go doesn't have a built-in "**reference**" keyword like C++, you can achieve **Call by Reference** manually by utilizing **Pointers**.

- **How it works:** Instead of passing the data itself, you pass its memory address using the address-of operator (`&x`). The function is built to accept a pointer (e.g., `*int`). If the function modifies the dereferenced value (`*y = *y + 1`), it is physically altering the original `x` stored in memory.
- **The Advantage (Blazing Speed):** No matter how large the underlying data structure is, a pointer is always just a tiny, fixed-size memory address (usually 8 bytes). Passing a pointer takes near-zero time and avoids unnecessary memory duplication.
- **The Disadvantage (Mutation Risk):** You completely lose encapsulation. If the called function accidentally deletes or corrupts the data, your original variable is permanently destroyed across the entire application.

Expert Takeaway:

When designing scalable backend architectures, deciding between Value and Reference is a daily engineering choice.

- If a function just needs to perform a quick calculation using a simple integer (like an ID) or a small string, pass it by **Value** to keep your state pure and safe from unintended side effects.
- If a function needs to process or update a massive, deeply nested JSON payload (like a full user profile or a large configuration object) from an incoming API request, pass it by **Reference** (as a pointer). This completely prevents the server from choking on memory by continually cloning huge objects as they travel through your application's middleware.